OptiPIM at Reduced Scale: Reproducing ILP-Optimal Processing-in-Memory Mappings

Group 12: Pari Khandelwal, Ashutosh Kumar, Parth Sandeep Maliye, Keshav Kaushik
Memory Centric Computer Architecture
Department of Computer Science and Engineering, IIT Guwahati

***Abstract*—Processing-in-memory (PIM) accelerators only pay off when each operator is mapped well onto banks, DRAM columns and time, and when its data is laid out to match. OptiPIM (ISCA 2025) casts this mapping and layout problem as an integer linear program (ILP) and reports large speedups over hand-written heuristics. We reproduce its core result at reduced scale with the authors' public flow and Gurobi: five single operators (three ResNet-50 convolutions, two BERT matrix multiplications) on HBM-PIM and SIMDRAM, scored with the authors' analytical model and replayed in their Ramulator2 simulator. OptiPIM beats the weight-parallel heuristic by 27–34× on HBM-PIM convolutions and by 1.1–2.5× on SIMDRAM, matching the paper's trend. It loses on one HBM-PIM matrix multiplication (0.78×), which we trace to the mapping using 24 of 32 banks; the ILP objective there is two orders of magnitude below the analytical cost; replaying both mappings in the simulator confirms the loss (0.79×). The analytical model tracks the simulator within 2.7% on HBM-PIM but underestimates SIMDRAM by 13–116%. We also found that `pim-opt` writes stride and dilation in the opposite order to the authors' scorer. Finally, we address OptiPIM's perfect-factorization constraint through a zero-padding extension for awkward loop bounds, which, for ViT attention, gives 1.71× (analytical model) and 1.40× (simulator) on SIMDRAM, and 10.97× (model) and 10.92× (simulator) on HBM-PIM.**

I. Introduction

PIM architectures place compute next to (HBM-PIM [3]) or inside (SIMDRAM [4]) DRAM arrays, so a deep-learning operator runs fast only if its loops are split well across banks, columns and time steps, and its inputs, weights and outputs are placed where the compute units can use them. Existing PIM software uses heuristics or exhaustive or random search, and mappers built for ASIC accelerators (e.g. COSA [5]) ignore the data-layout costs that dominate PIM.

OptiPIM [1] models mapping and data layout together as one ILP, solved to optimality by Gurobi in seconds to minutes, and reports average speedups of 2.3× on SIMDRAM and 72.8× on HBM-PIM over the weight-parallel heuristic. This report reproduces that comparison at the reduced scale set for the course, checks the paper's analytical cost model against the simulator, and addresses the perfect-factorization bottleneck through zero-padding.

Our contributions are: (1) a working build of the authors' flow on a WSL2 laptop with Gurobi 13, including a workaround for a presolve failure; (2) optimal mappings for five operators on both targets and a comparison with the authors' heuristic mappings, with a reproducible scoring and plotting pipeline; (3) a three-way comparison of ILP objective, analytical model and simulator that exposes where they disagree; (4) an automated zero-padding framework that mitigates OptiPIM's perfect-factorization bottleneck, achieving, on ViT attention, 1.71× (model) / 1.40× (simulator) on SIMDRAM and 10.97× (model) / 10.92× (simulator) on HBM-PIM.

II. Background: the OptiPIM Formulation

Targets. HBM-PIM places a SIMD unit next to each pair of banks; its cost is dominated by moving inputs and weights into the unit's registers. SIMDRAM computes bit-serially inside the DRAM array, so multiplication and addition latency dominate.

Mapping space. Each loop of an operator (N, K, P, Q, C, R, S for a 2D convolution; N, K, P, Q, R for a matrix multiplication) is split into three factors whose product is the loop bound: one across banks (level 2, parallel), one across DRAM columns (level 1, parallel) and one over time (level 0). OptiPIM also chooses *transformation coefficients* that decide how the indices are laid out in memory (flexible indexing), which lets the solver trade parallelism against input reuse.

Cost model. The paper derives analytical expressions for computation, output reduction and input loading (Eq. 17–19) and minimises their sum (Eq. 20), estimating the number of unique inputs with its own counting method (Eq. 11). The authors validate their model against a modified Ramulator2 and report $R^2 > 0.99$ on both targets (Fig. 9 of [1]). The formulation assumes that every loop bound factorises exactly across the three levels (Sec. 5.1.1).

III. Implementation and Experimental Setup

A. Toolchain

We use the authors' public flow (ETHZ-DYNAMO/OptiPIM [2], commit 9be8f9a), forked into our group repository. It has three parts: `pim-opt`, an MLIR pass built on LLVM (commit 39048b69) that reads one layer in the `linalg` dialect, builds the ILP, calls Gurobi and writes the mapping as a trace file; a Python analytical model (`validation/analytical_modeling`) that scores a trace with the paper's cost model; and a modified Ramulator2 [7] that replays a trace as PIM commands and reports `memory_system_cycles`.

TABLE I
OPERATORS USED IN OUR EVALUATION

ID	Source layer	Shape
C1	ResNet-50 conv12	3×3, 128→128 ch., 28×28 out, stride 2
C2	ResNet-50 conv2	3×3, 64→64 ch., 56×56
C3	ResNet-50 conv1	1×1, 64→64 ch., 56×56
M1	BERT fc0	GEMM 128×768×768
M2	BERT fc1	batched GEMM 12×128×64×128

TABLE II
GUROBI SOLVE TIME PER OPERATOR (SECONDS)

Operator	HBM-PIM	SIMDRAM
C1	64.2	44.3
C2	181.0	85.0
C3	27.5	5.8
M1	2.9	2.4
M2	5.8	1.1

We built the flow on Ubuntu 22.04 under WSL2 on a Windows laptop (6 logical cores and 11 GB of RAM given to WSL, g++ 11.4.0). Building LLVM, `pim-opt` and the simulator took about 4 h 15 min. We used Gurobi 13.0.3 [8] with a free academic licence; the paper used 11.0.3. The repository's CMake module only searches for Gurobi 11.0 and 12.0, so we exposed the 13.0 library under the 12.0 name; no source code was changed. With Gurobi 13, presolve wrongly reported the 3×3 convolutions C1 and C2 infeasible on HBM-PIM at 32 banks (the same models solve at 8, 16, 17 and 20 banks), so we solve with presolve disabled (`Presolve 0`), which gives a proven optimum. A Dockerfile in the repository reproduces the build for members on macOS.

B. Operators and targets

Following the reduced scope, we map five single operators rather than full networks (Table I), taken from the layer files in `nn_models/` with the bank count set to 32. C1 is the layer the paper analyses in its Fig. 12. Each operator is mapped onto HBM-PIM (`--target-device-type 1, data/hbm_pim.json`) and SIMDRAM (`--target-device-type 0, data/dram.json`), with 16-bit values.

C. ILP configuration and solve time

All runs use the paper's full method: flexible indexing (`--trans-coeff-method 4`), the paper's input-counting estimate (`--number-counting-method 0`) and the PIM cost objective (`--obj-method 0`). One script solves all ten operator–target pairs, stores each mapping and Gurobi log, and records solve times (Table II). Every run ended with "Optimal solution found". Solve times of 1.1–181 s are in line with the paper's 1–209 s on a consumer-grade 16-core CPU.

D. Baseline: weight-parallel heuristic

The paper's baseline (Sec. 6.4.1) tiles the weight matrix or convolution filters to the SIMD width and spreads the tiles over banks, columns and rows, reloading inputs for every SIMD operation. We use the heuristic mappings the authors ship in `baseline_mappings/heuristic_1` (and `heuristic_2` as a cross-check) for exactly our five layers at 32 banks (ResNet50-12, -2, -1 and BERT-0, -1; problem sizes checked against our operators). The `heuristic_1` mapping for C1 is identical to the paper's Fig. 12(c). Because the authors ship only trace files, not code, Member 2 re-implemented the heuristic (`heuristic.py`, covering convolution, matrix multiplication and batched matrix multiplication) and wrote `check_heuristic.py`, which compares the generated traces with the shipped ones. All ten traces (five operators on two targets) match the shipped `heuristic_1` files exactly and reproduce their analytical costs.

E. Scoring

Every mapping, OptiPIM's and the heuristic's, is scored with the same analytical model and arch file, so speedups are unit-free. Absolute costs are never compared across targets, because the two use different formulas and arch files and we have not verified that both are cycles of the same clock. Our script `group12/analysis/rerun.sh` rescores all traces and regenerates every figure and table in this report from the committed files.

While building this pipeline we found that `pim-opt` writes the `DilationStride` line of a trace as dilation, dilation, stride, stride (`DataLayoutMILP.cpp`), whereas the authors' Python parser reads the first field as stride and the third as dilation, which is also the order of their shipped heuristic traces. Scored directly, C1 (stride 2) is treated as a stride-1, dilation-2 convolution. Our scorer swaps the fields for `pim-opt` traces. The fix leaves C1 on HBM-PIM unchanged (870,912) and raises C1 on SIMDRAM from 24,995,102 to 26,701,846.

IV. BASELINE RESULTS COMPARED WITH THE PAPER

A. OptiPIM versus the heuristic

Fig. 1 and Table III compare OptiPIM with `heuristic_1`. On HBM-PIM, OptiPIM is 27–34× faster on the three convolutions and 2.8× on M2; on SIMDRAM it is 1.1–2.5× faster on every operator (geometric mean 1.74×). This matches the paper's trend: very large wins on HBM-PIM convolutions, where the heuristic breaks input reuse, and modest wins on SIMDRAM, where bit-serial compute dominates every mapping (paper: 72.8× and 2.3× on average over full networks, 2.4× for BERT on HBM-PIM). Our averages are over five operators only, so they are not directly comparable to the paper's network-level numbers.

Where OptiPIM loses: M1 on HBM-PIM. An ILP solved to optimality cannot lose to a heuristic if both are judged by the ILP's own cost, yet under the analytical model OptiPIM's M1 mapping is 1.27× slower than the heuristic's. The simulator agrees: it needs 1,082,667 cycles for the OptiPIM mapping and 853,291 for the heuristic's, a ratio of 0.79× (model: 0.78×). The cost breakdown pinpoints the reason: the OptiPIM mapping uses 24 of the 32 banks, while the heuristic uses all 32. Its compute term is identical to the heuristic's (147,456),

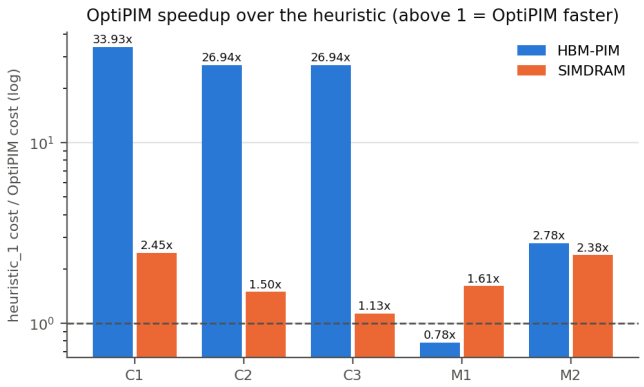


Fig. 1. OptiPIM speedup over the weight-parallel heuristic (`heuristic_1` cost / OptiPIM cost, analytical model, log scale).

TABLE III
ANALYTICAL COST OF OPTIPIM AND HEURISTIC MAPPINGS, AND SPEEDUP

	HBM-PIM			SIMDRAM		
Op	OptiPIM	Heur.	×	OptiPIM	Heur.	×
C1	870,912	29,546,496	33.9	26,701,846	65,549,429	2.45
C2	1,096,704	29,546,496	26.9	24,338,662	36,395,701	1.50
C3	121,856	3,282,944	26.9	3,575,824	4,053,173	1.13
M1	1,064,960	835,584	**0.78**	15,443,838	24,881,280	1.61
M2	221,184	614,400	2.78	3,257,727	7,769,472	2.38

and its input loading (524,288 vs 393,216) and weight loading (393,216 vs 294,912) are both exactly $32/24 = 4/3$ larger, which accounts for the whole gap. The ILP still preferred this mapping because its HBM-PIM objective for M1 is only 9,888, about 108× below the analytical cost (Section IV-E), so it does not see the loading costs that the extra banks would have reduced. We have not confirmed in the ILP source which terms are missing.

B. Cost breakdown

Fig. 2 splits each analytical cost into the terms the model adds up. On HBM-PIM, input loading is 85–98% of the heuristic's cost on every operator except M1; OptiPIM cuts it to 26–59% of a much smaller total, at the price of more weight loading. On SIMDRAM, bit-serial compute is 74–94% of the cost for both mappers. Both observations match the paper's Fig. 10(b,d): compute dominates SIMDRAM and data loading dominates HBM-PIM.

C. Mapping case study: ResNet-50 Conv12 (C1)

On HBM-PIM our ILP maps C1 with Q=2 and C=16 across the 32 banks, P=28, Q=2, C=8 across columns, and K=128, Q=7, R=S=3 in time (Table IV). The heuristic instead puts K across banks (K_2=32, K_1=4) and reloads the input for every filter (C_0=128), so its input loading is 28,901,376 against OptiPIM's 225,792, which accounts for almost all of the 34× gap. The paper's Fig. 12 reports about 100× for this layer with a different OptiPIM mapping that

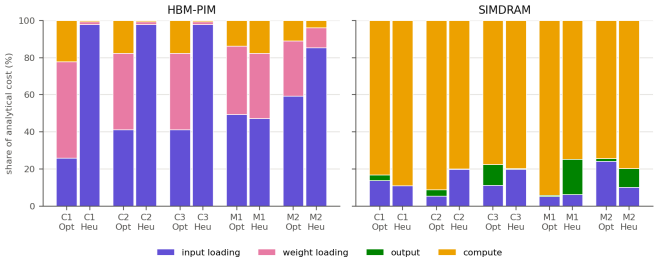


Fig. 2. Share of each cost term in the analytical model, OptiPIM (Opt) vs `heuristic_1` (Heu). On HBM-PIM "compute" is the row-activation term; on SIMDRAM it is bit-serial multiply and add.

TABLE IV
C1 (RESNET-50 CONV12) LOOP SPLIT FACTORS: OPTIPIM VS. HEURISTIC

		OptiPIM (32 banks)			Heuristic (32 banks)		
Loop	Dim	$L2$ (Bank)	$L1$ (Col)	$L0$ (Time)	$L2$ (Bank)	$L1$ (Col)	$L0$ (Time)
N (Batch)	1	1	1	1	1	1	1
K (Out Ch)	128	1	1	128	32	4	1
P (Out H)	28	1	28	1	1	28	1
Q (Out W)	28	2	2	7	1	4	7
C (In Ch)	128	16	8	1	1	1	128
R (Kernel H)	3	1	1	3	1	1	3
S (Kernel W)	3	1	1	3	1	1	3
Total		**32 banks**	**448 cols**	**8,064**	**32 banks**	**448 cols**	**8,064**

uses 16 banks. We rebuilt the paper's Fig. 12(b) mapping as a trace (its R and S coefficients are not shown, so we set them as in our mapping) and scored it: 3,161,088, i.e. 9.3× better than the heuristic and 3.6× worse than our 32-bank mapping. Its output-reduction term (6,272) equals the value printed in the paper, but its input loading (903,168) does not (18,432), so the costs in the paper's Fig. 12 appear to be in different units from the Python model; we therefore compare ratios only.

D. Analytical model versus simulator

We replayed the OptiPIM and the heuristic mapping of all five operators on both targets (20 mappings) in Ramulator2 (Fig. 4, Table V). On HBM-PIM the model is within 2.7% of the simulator for all ten mappings replayed, consistent with the paper's Fig. 9. On SIMDRAM the model underestimates the simulator for every mapping, by 13% to 116%, and by different amounts for the two mappers (e.g. 41% for OptiPIM but 13% for the heuristic on M1, but 97% against 33% on C1), so the error is not a constant factor. The largest gaps are the two 3×3 convolutions: the simulator reports the same 52,540,980 cycles for C1 and C2, matching the identical ILP objective of their mappings (23,273,702), while the model scores C1 10% higher than C2. The authors' own validation data (`exp_results/fig9`) agrees within a few percent on SIMDRAM, so the gap is specific to our mappings or our simulator build; we did not find its cause.

The simulator also lets us check the speedups of Table III. On HBM-PIM they agree: 33.0× (C1), 26.4× (C2), 26.4× (C3), 0.79× (M1) and 2.75× (M2) against 33.9×, 26.9×, 26.9×, 0.78× and 2.78× from the model. On SIMDRAM OptiPIM is still faster for all five operators, but by 1.66×

TABLE V
ANALYTICAL MODEL VERSUS RAMULATOR2 (CYCLES)

Op, mapper	Target	Model	Simulator	Diff.
C1 OptiPIM	HBM-PIM	870,912	894,779	+2.7%
C2 OptiPIM	HBM-PIM	1,096,704	1,120,256	+2.1%
C1 heuristic	HBM-PIM	29,546,496	29,529,983	−0.1%
C2 heuristic	HBM-PIM	29,546,496	29,529,983	−0.1%
C3 OptiPIM	HBM-PIM	121,856	124,464	+2.1%
C3 heuristic	HBM-PIM	3,282,944	3,281,117	−0.1%
M1 OptiPIM	HBM-PIM	1,064,960	1,082,667	+1.7%
M1 heuristic	HBM-PIM	835,584	853,291	+2.1%
M2 OptiPIM	HBM-PIM	221,184	224,061	+1.3%
M2 heuristic	HBM-PIM	614,400	616,797	+0.4%
C1 OptiPIM	SIMDRAM	26,701,846	52,540,980	+96.8%
C2 OptiPIM	SIMDRAM	24,338,662	52,540,980	+115.9%
C1 heuristic	SIMDRAM	65,549,429	87,405,433	+33.3%
C2 heuristic	SIMDRAM	36,395,701	59,496,616	+63.5%
C3 OptiPIM	SIMDRAM	3,575,824	4,919,252	+37.6%
C3 heuristic	SIMDRAM	4,053,173	6,612,996	+63.2%
M1 OptiPIM	SIMDRAM	15,443,838	21,804,288	+41.2%
M1 heuristic	SIMDRAM	24,881,280	28,097,818	+12.9%
M2 OptiPIM	SIMDRAM	3,257,727	5,725,248	+75.8%
M2 heuristic	SIMDRAM	7,769,472	10,739,994	+38.2%

TABLE VI
ILP OBJECTIVE AND ANALYTICAL COST OF THE OPTIPIM MAPPINGS

	HBM-PIM		SIMDRAM	
Op	ILP	Model	ILP	Model
C1	360,149	870,912	23,273,702	26,701,846
C2	364,316	1,096,704	23,273,702	24,338,662
C3	45,906	121,856	3,174,416	3,575,824
M1	9,888	1,064,960	14,632,830	15,443,838
M2	3,040	221,184	2,471,295	3,257,727

(C1), 1.13× (C2), 1.34× (C3), 1.29× (M1) and 1.88× (M2) instead of 2.45×, 1.50×, 1.13×, 1.61× and 2.39× (geometric mean 1.44× instead of 1.74×), so the model gets the direction right but not the size of the gain.

E. Solver objective versus analytical model

Table VI compares the ILP objective with the analytical cost of the same mapping. On SIMDRAM they agree within 5–32%. On HBM-PIM the model's cost is 2.4–3.0× the objective for the convolutions and 73–108× for the matrix multiplications. `pim-opt` also prints its own "Real Cost" after solving (176,544 for M1 on HBM-PIM), which matches neither. The objective is therefore not a cycle count, and on HBM-PIM matrix multiplications it ranks mappings differently from the model that the simulator confirms, which is how the M1 loss arises.

F. Summary of differences from the paper

- HBM-PIM M1: OptiPIM 0.78× the heuristic because it uses 24 of 32 banks; the ILP objective does not reflect the loading costs (above).
- SIMDRAM model-vs-simulator gap of 13–116%, not seen in the paper, and different for the two mappers, so the model also misjudges the size of the SIMDRAM speedups.
- Smaller C1 gain (34× vs about 100×); the paper's Fig. 12 costs are in different units, and its mapping uses 16 banks.
- Gurobi 13.0.3 instead of 11.0.3, with presolve disabled for HBM-PIM (see Sec. VI); with several optimal solutions a different version may return a different mapping.
- Five single operators instead of whole networks, so we cannot reproduce the paper's per-network averages.

V. OUR CONTRIBUTION: ZERO-PADDING FOR NON-SMOOTH LOOP BOUNDS

Motivation. OptiPIM assumes "a perfect factorization of the original loop bound" (Sec. 5.1.1), selecting loop bounds $L_{x,l}$ strictly from the exact divisors of L_x. While effective for legacy CNN layers whose channel and feature dimensions are multiples of powers of two, modern architectures frequently feature prime or non-smooth dimensions. For instance, Vision Transformer (ViT-B/16) attention matrices operate over sequence lengths of 197 tokens (14×14 patches plus class token), where 197 is prime. Similarly, GPT-2's vocabulary projection evaluates a dimension of $R = 50{,}257 = 29 \times 1733$. Because prime bounds admit only trivial divisors (1 and itself), the ILP cannot split the dimension across both banks (Level 2) and columns (Level 1). Consequently, PIM hardware with 32 banks and 8,192 SIMDRAM columns suffers severe internal fragmentation and idle compute lanes.

Zero-Padding Methodology. To address this limitation without modifying the underlying compiler pass or ILP formulation, we propose an automated semantics-preserving zero-padding exploration framework (`pad_search.py`). The framework pads awkward loop bounds to nearby 7-smooth numbers (integers whose prime factors do not exceed 7, e.g., $197 \rightarrow 200$, $50{,}257 \rightarrow 50{,}400$). Zero-padding is strictly semantics-preserving: padded output elements are computed on zeroed inputs and discarded upon host gathering, while padded reduction dimensions accumulate zeros. We constrain candidates to $\leq$25% growth per dimension and $\leq$30% overall multiply-accumulate (MAC) overhead. For each candidate, the script synthesizes a minimal MLIR kernel, invokes `pim-opt` to find the optimal mapping, and re-scores the resulting trace using the authors' validated Python analytical cost model. Because the unpadded operator is always evaluated as a candidate, the mapping the search selects is never worse than baseline OptiPIM under the analytical model.

Results and Discussion. Table VII and Figure 5 present the results across all evaluated layers. Our findings confirm three key insights:

1) *Prime Sequence Lengths (Transformers):* For ViT Attention QK ($12 \times 197 \times 64 \times 197$), padding R from 197 to 200 (only 1.52% extra MACs) yields a **1.71× speedup** on SIMDRAM. On HBM-PIM, padding P to 200 yields an even more dramatic **10.97× speedup** (reducing cycles from 2.87×10^6 to 2.62×10^5). A likely reason is that $P = 197$ is prime, so the ILP cannot split it across banks and columns, whereas $200 = 8 \times 25$ admits rich factorizations (we did not inspect the mappings in detail). Replaying both mappings in Ramulator2 gives 2.99×10^6 and 2.73×10^5

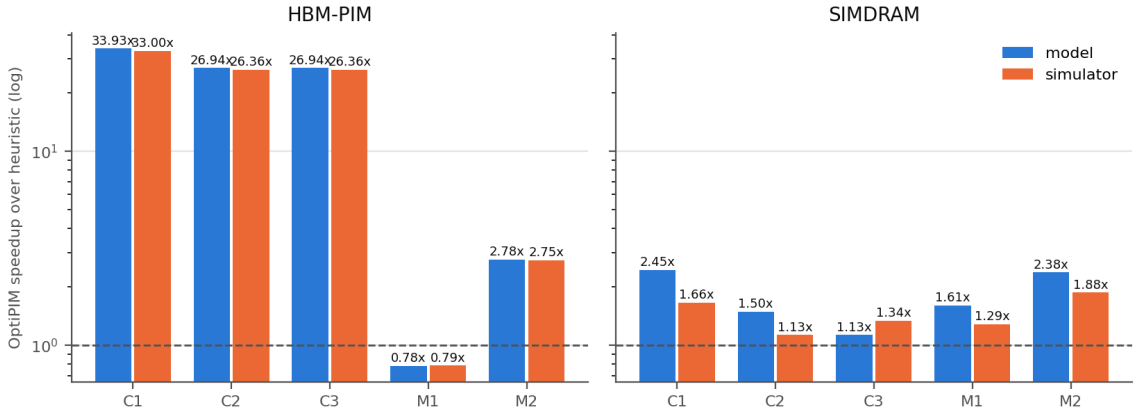


Fig. 3. OptiPIM speedup over the heuristic according to the analytical model and to the Ramulator2 simulator (log scale).

TABLE VII
ANALYTICAL EVALUATION OF ZERO-PADDING ON 8 OPERATORS AND BOTH TARGETS (32 BANKS). COSTS ARE ANALYTICAL-MODEL CYCLES; ONLY THE THREE PAIRS IN TABLE VIII WERE ALSO SIMULATED.

Operator	Target	Unpadded (cyc)	Best variant	Overhead	Speedup
ViT Attention QK	SIMDRAM	1.14×10^7	pad_R200	+1.5%	**1.71**×
ViT Attention QK	HBM-PIM	2.87×10^6	pad_P200	+1.5%	**10.97**×
ViT QKV Proj.	SIMDRAM	9.44×10^7	pad_P200	+1.5%	**1.28**×
ViT QKV Proj.	HBM-PIM	5.75×10^6	pad_P200	+1.5%	**1.84**×
GPT-2 LM Head	SIMDRAM	9.23×10^6	pad_R50400	+0.3%	**1.15**×
GPT-2 LM Head	HBM-PIM	2.40×10^5	pad_R51450	+2.4%	**1.08**×
AlexNet conv0	HBM-PIM	2.75×10^6	pad_K70_P56_Q56	+13.4%	**3.79**×
AlexNet conv0	SIMDRAM	Infeasible	pad_K70_P56_Q56	+13.4%	Feasible
AlexNet conv1	HBM-PIM	1.54×10^6	unpadded	0.0%	1.00×
AlexNet conv1	SIMDRAM	4.64×10^7	unpadded	0.0%	1.00×
AlexNet conv2	HBM-PIM	8.23×10^5	pad_K392	+2.1%	1.19×
AlexNet conv2	SIMDRAM	2.25×10^7	unpadded	0.0%	1.00×
AlexNet fc0	HBM-PIM	2.40×10^5	pad_R4116	+0.5%	1.01×
AlexNet fc0	SIMDRAM	7.93×10^6	unpadded	0.0%	1.00×
ResNet-50 conv5	HBM-PIM	8.71×10^5	pad_K525	+2.5%	1.08×
ResNet-50 conv5	SIMDRAM	2.43×10^7	unpadded	0.0%	1.00×

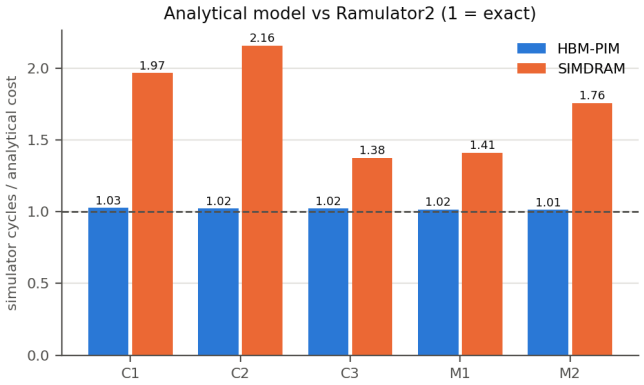


Fig. 4. Simulator cycles divided by analytical cost for OptiPIM mappings (1 = exact agreement). The heuristic mappings are in Table V.

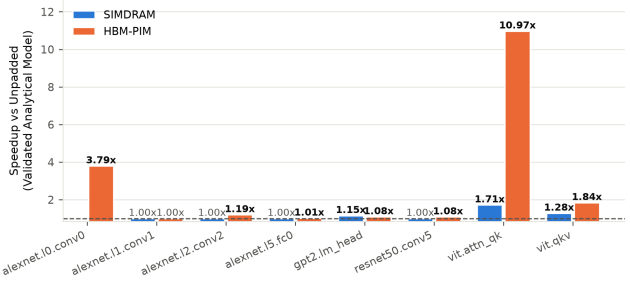


Fig. 5. Speedup of optimal zero-padded mappings over unpadded OptiPIM across target operators under the validated analytical model.

cycles, i.e. 10.92×, so the model is right that the unpadded mapping is slow. The ILP objective hides this: pim-opt's own cost for the unpadded mapping is only 9.5×10^4 against an analytical cost of 2.87×10^6, the same objective-versus-model effect that made OptiPIM lose on M1. Part of the gain is therefore OptiPIM's weak HBM-PIM objective rather than padding alone. Similarly, ViT QKV projection achieves a **1.84**× speedup on HBM-PIM and **1.28**× on SIMDRAM in

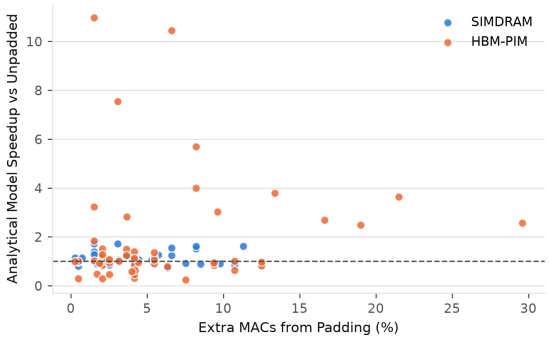


Fig. 6. Trade-off between MAC computation overhead and analytical speedup across all candidate padded mappings.

TABLE VIII
PADDING SPEEDUP: ANALYTICAL MODEL VERSUS RAMULATOR2 (CYCLES, UNPADDED / PADDED)

Operator, target	Variant	Sim. cycles	Model	Sim.
ViT attn QK, HBM-PIM	pad_P200	$2.99\times10^6/2.73\times10^5$	10.97×	10.92×
ViT QKV, HBM-PIM	pad_P200	$5.86\times10^6/3.19\times10^6$	1.84×	1.83×
ViT attn QK, SIMDRAM	pad_R200	$1.29\times10^7/9.23\times10^6$	1.71×	1.40×

the model. We replayed three of these pairs in Ramulator2 (Table VIII): the HBM-PIM gains are confirmed to within 0.5% (10.92× and 1.83×), while the SIMDRAM gain for ViT attention is 1.40× in the simulator against 1.71× in the model. This is consistent with the SIMDRAM model-versus-simulator gap reported above, so for SIMDRAM we regard the simulator figure as the more reliable one. The other rows of Table VII are model-only.

2) *Large Vocabularies (Language Models):* For GPT-2's vocabulary projection ($R = 50{,}257$), padding R to the 7-smooth dimension $50{,}400$ adds just 0.28% overhead while delivering a **1.15× speedup** on SIMDRAM, where large column counts require smooth divisibility.

3) *Feasibility Recovery and Controls:* For AlexNet conv0 on SIMDRAM, pim-opt (with Gurobi's default presolve) reported no feasible mapping for the unpadded 55×55 shape, whereas eight padded variants solved. We did not investigate the cause further. Conversely, on control CNN layers with power-of-two channel counts (AlexNet conv1, conv2, fc0, and ResNet-50 conv5), padding provided no benefit on SIMDRAM (1.00×) and only 1.01–1.19× on HBM-PIM (model only), showing that the framework safely retains the exact unpadded solution whenever factorization is already smooth. Figure 6 shows that overhead alone does not predict the outcome. Of the 112 scored candidates, the median speedup is 1.00× both for variants with at most 5% extra MACs (21 of 66 are more than 5% slower than unpadded) and for 5–15% (median 0.93×). The large gains come from a few variants that make a bound factorizable, and the four variants above 15% overhead all gained (up to 3.6×). A search over several candidates is therefore needed, rather than always padding to the nearest smooth size.

VI. LIMITATIONS OF OUR SETUP

- **No real hardware.** All cycle counts come from the analytical model and the authors' Ramulator2-based simulator.
- **Few simulator points.** We simulated 20 mappings (two per operator and target), against the paper's 12,000 random mappings, and one run each, so we cannot report an R^2 comparable to the paper's.
- **Unexplained gaps.** The SIMDRAM model-vs-simulator gap and the ILP-objective-vs-model gap on HBM-PIM are measured but not explained.
- **Padding evaluation.** Only three padded mappings were simulated, the other padding results are model-only, and padded shapes were not compared with the heuristic baseline.
- **Different solver version and machine.** Gurobi 13.0.3 with presolve disabled for HBM-PIM (the logs show it disabled for four of five SIMDRAM operators and at its default for SIMDRAM M1; the padding runs used the default on SIMDRAM), on a laptop rather than the paper's setup; absolute solve times are not comparable.
- **Five operators, one configuration.** 32 banks and 16-bit values only.
- **Heuristic baseline.** We use the authors' shipped heuristic mappings rather than the original HBM-PIM software.

VII. WHAT WE LEFT OUT

TABLE IX
PARTS OF THE PAPER LEFT OUT AND WHAT WE USED INSTEAD

Paper	Our setup
Real-hardware measurements	Analytical model and Ramulator2 simulator
PIM-DL [6] genetic-search baseline	Weight-parallel heuristic only
Full networks (AlexNet, ResNet, VGG, UNet, BERT, Llama)	Five single operators from ResNet-50 and BERT
Memory scalability study	32 banks only
PyTorch → Torch-MLIR front end	The authors' shipped `.linalg.mlir` layer files
COSA-style ASIC mapping baseline	Not run

VIII. CONCLUSION

With the authors' flow and Gurobi 13 we reproduced OptiPIM's main trend at reduced scale: large wins over the weight-parallel heuristic on HBM-PIM convolutions and modest wins on SIMDRAM. The reproduction also exposed three weak points that the paper's averages hide: a matrix multiplication where the ILP picks a mapping slower than the heuristic because its objective diverges from the validated cost model, a SIMDRAM model-vs-simulator gap, and a stride/dilation ordering mismatch between `pim-opt` and its scorer. Our zero-padding extension further shows that relaxing OptiPIM's exact divisor constraint can give large gains on workloads with prime token counts: for ViT attention, 1.40× on SIMDRAM and 10.92× on HBM-PIM in the simulator

(1.71× and 10.97× in the model). Gains on other operators are smaller and were checked with the model only.

Member Contributions

Pari Khandelwal (Member 1) built the toolchain (LLVM, `pim-opt`, Ramulator2) and the Gurobi setup on WSL2, diagnosed the Gurobi 13 presolve failure, solved all operators on both targets, wrote the scoring, simulator and plotting scripts, found the stride/dilation mismatch, wrote the Dockerfile and build instructions, and wrote the setup, results and limitations sections. **Ashutosh Kumar** (Member 2) re-implemented the weight-parallel baseline heuristic (`heuristic.py`) and its checker (`check_heuristic.py`), which reproduces all ten shipped `heuristic_1` traces and their analytical costs exactly, and wrote the baseline-method notes. **Parth Sandeep Maliye** (Member 3) built the evaluation pipeline (`evaluate.py`, `results.csv`) that merges analytical costs and simulator cycles, and produced the speedup, cost-breakdown and model-versus-simulator correlation figures. **Keshav Kaushik** (Member 4) designed, implemented, and evaluated the dimension padding extension, created the candidate generator and analytical rescoring pipeline, and evaluated awkward transformer and CNN operators across both architectures.

Code

All source code, mappings, results and build instructions: https://github.com/parikhwal291/OptiPIM-grp12. `group12/README.md` explains the build; `bash group12/analysis/rerun.sh . . group12/results/plots` regenerates every figure and table in this report.

References

[1] J. Liu *et al.*, "OptiPIM: Optimizing processing-in-memory acceleration using integer linear programming," in *Proc. ISCA*, 2025.

[2] ETHZ-DYNAMO, "OptiPIM," GitHub repository. [Online]. Available: https://github.com/ETHZ-DYNAMO/OptiPIM

[3] S. Lee *et al.*, "Hardware architecture and software stack for PIM based on commercial DRAM technology," in *Proc. ISCA*, 2021.

[4] N. Hajinazar *et al.*, "SIMDRAM: A framework for bit-serial SIMD processing using DRAM," in *Proc. ASPLOS*, 2021.

[5] Q. Huang *et al.*, "CoSA: Scheduling by constrained optimization for spatial accelerators," in *Proc. ISCA*, 2021.

[6] C. Li *et al.*, "PIM-DL: Expanding the applicability of commodity DRAM-PIMs for deep learning via algorithm-system co-optimization," in *Proc. ASPLOS*, 2024.

[7] H. Luo *et al.*, "Ramulator 2.0: A modern, modular, and extensible DRAM simulator," *IEEE Comput. Archit. Lett.*, 2023.

[8] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024.